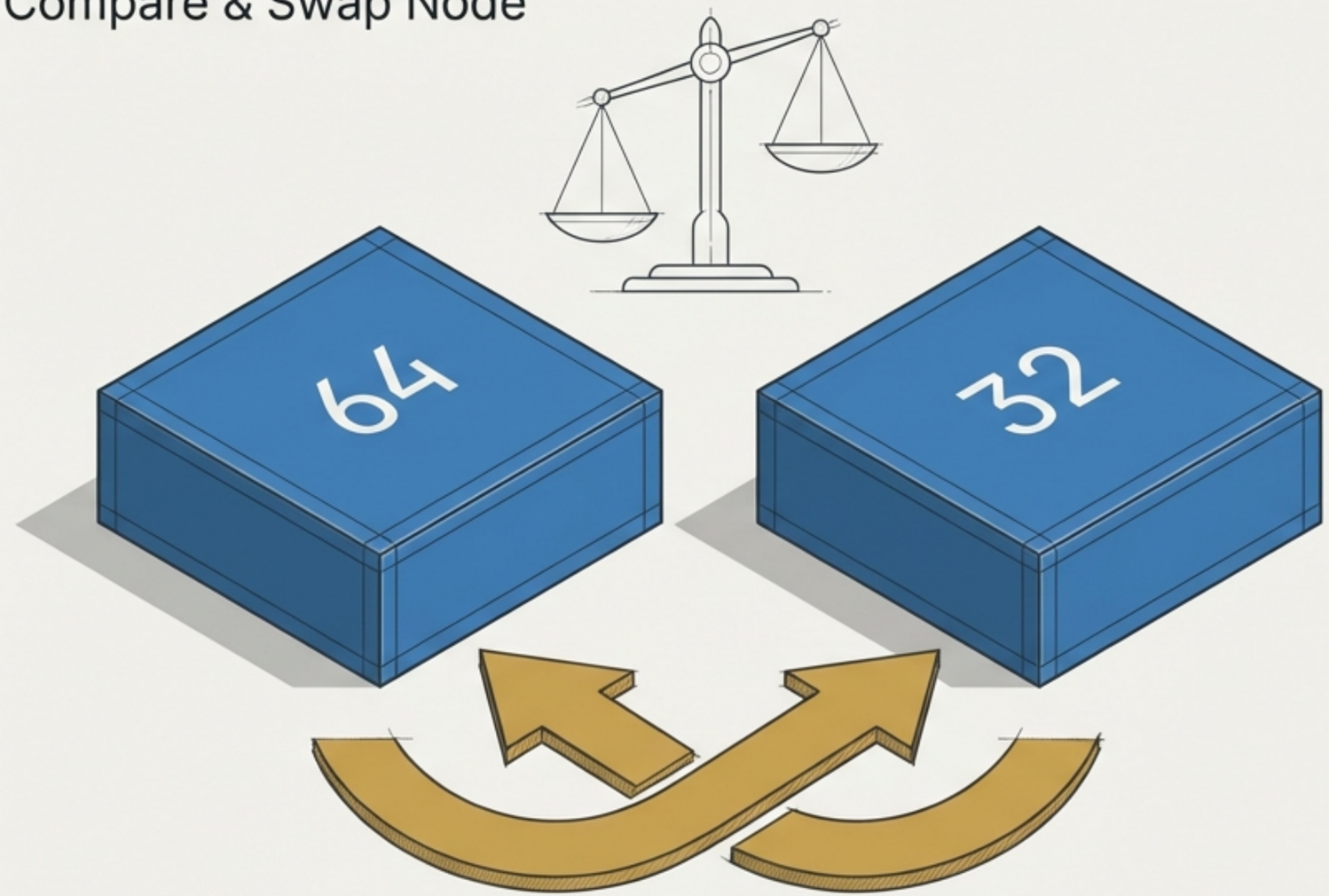
The Bubble Sort Blueprint

Transforming chaotic data sequences into ordered logic.

The fundamental micro-action is compare and swap

Compare & Swap Node



Rule

The left element must be strictly smaller than the right element.

Action

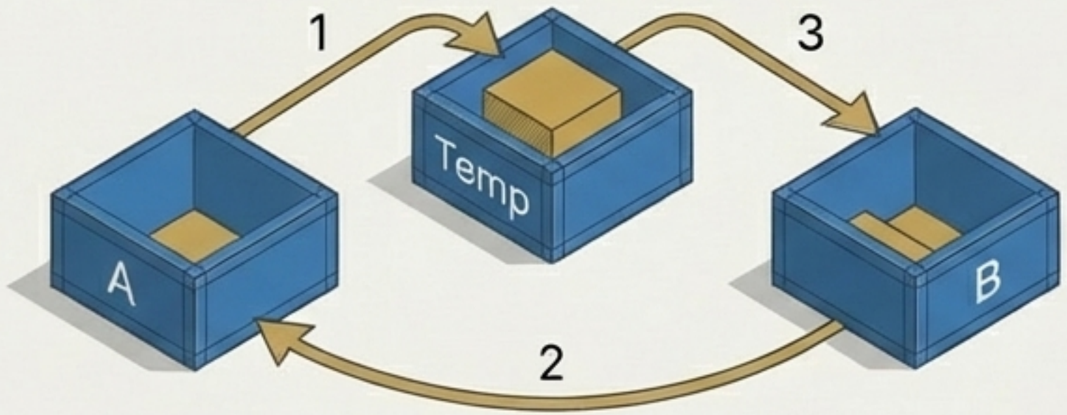
If the left is larger, their physical positions are instantly interchanged.

Progression

The algorithm steps sequentially through the array, assessing consecutive pairs (Index 0 & 1, then 1 & 2).

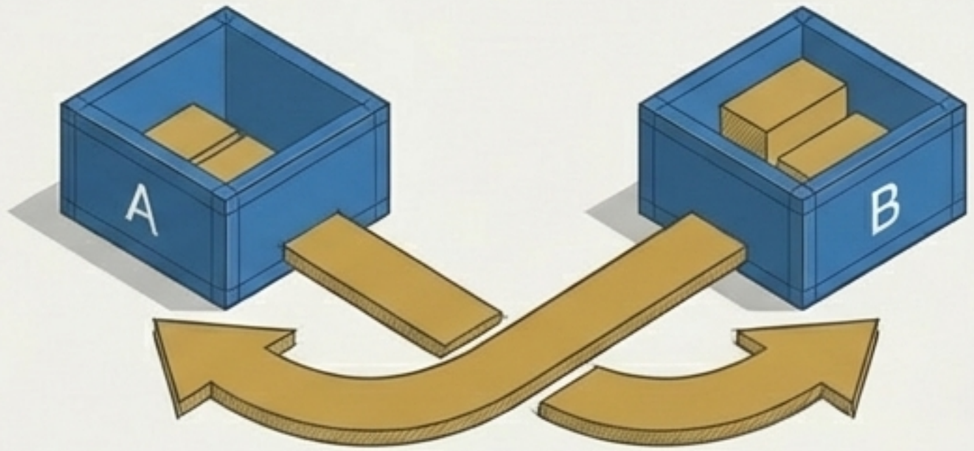
Python executes the swap in a single elegant instruction

The Traditional Paradigm (C++/Java)




```
Temp = A;
A = B;
B = Temp;
```

The Python Superpower

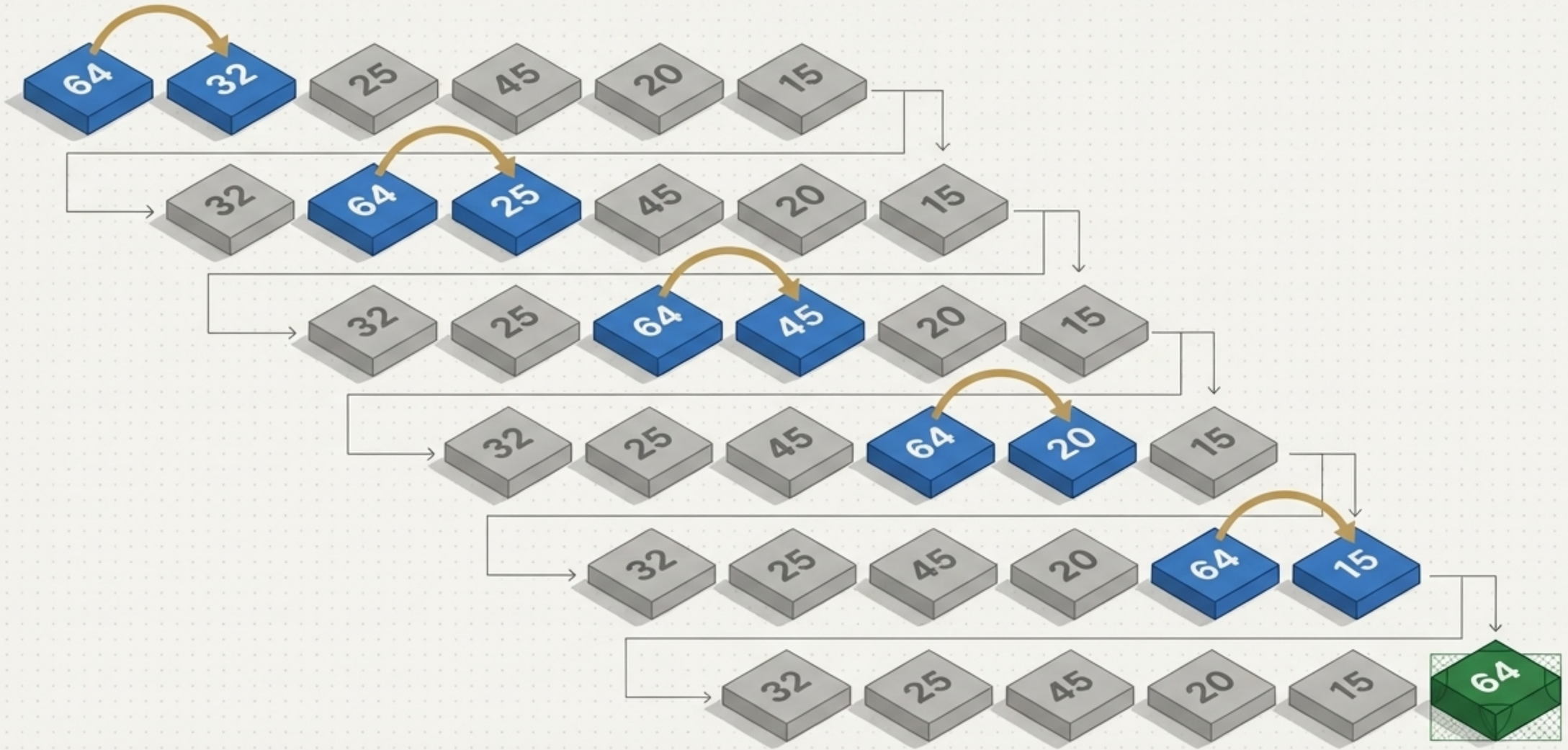



```
A, B = B, A
```

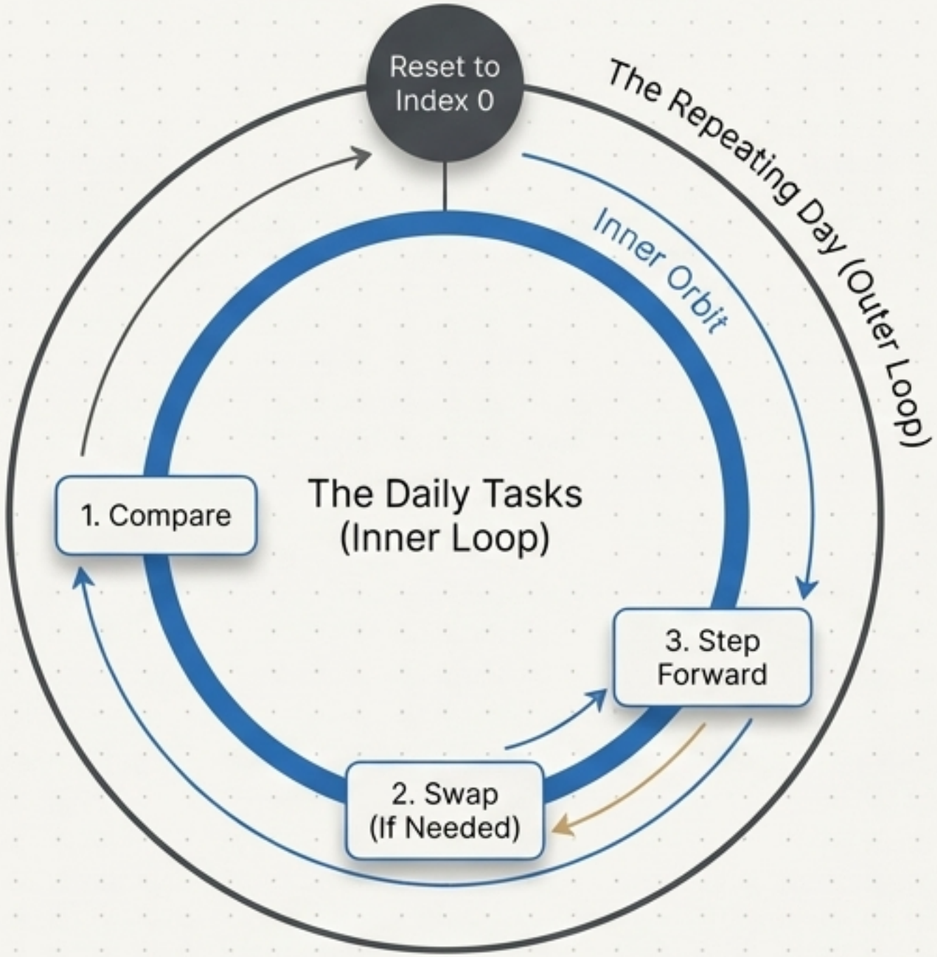
Python performs the entire mathematical interchange in a single, powerful line of syntax via tuple unpacking.

The largest element physically bubbles to the end of the array

Consecutive comparisons force the largest integer to the final index.



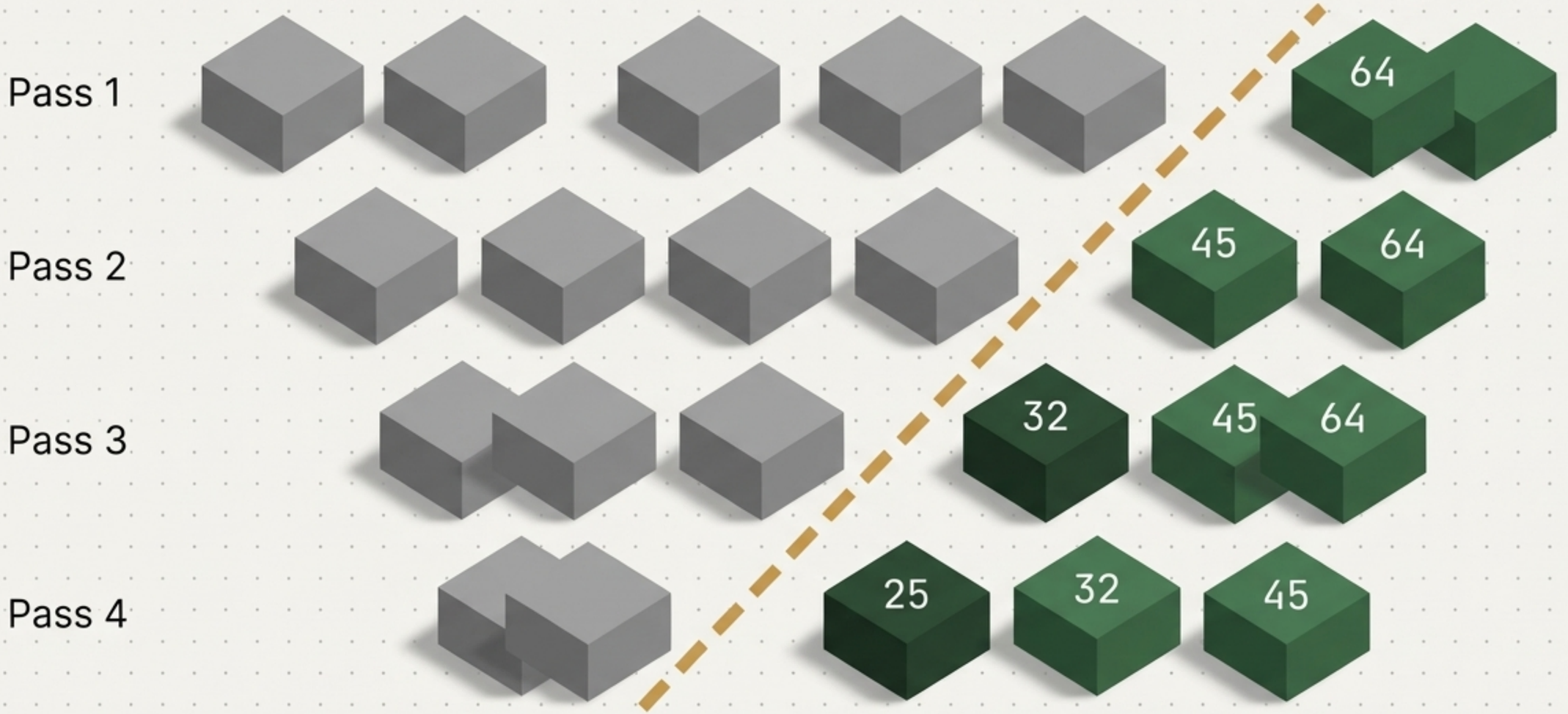
Iterations mirror a **daily repeating routine**



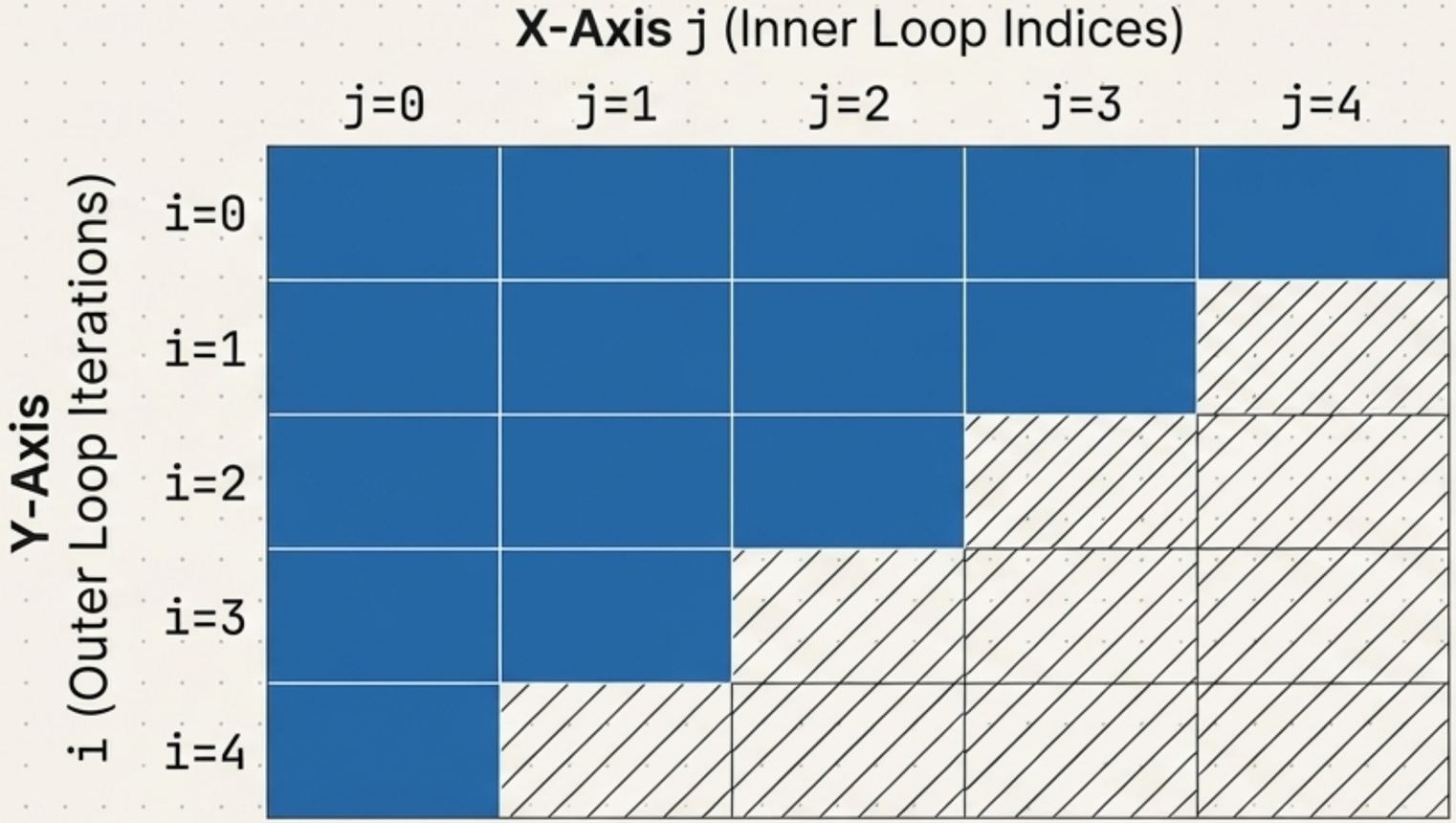
Just as a morning routine repeats daily with slight variations, the algorithm completes a full pass through the array's inner tasks, then utilizes an outer loop to repeat the entire cycle and sort the remaining elements.

The active sorting zone shrinks with every pass

Elements that reach their correct final position are mathematically removed from subsequent comparisons.



The mathematical relationship between loops forms a descending grid




```
j < n - 1 - i
```

Subtracting the variable ‘i’ mathematically prevents redundant comparisons on already-sorted elements, forming a shrinking optimization window.

The complete Bubble Sort blueprint

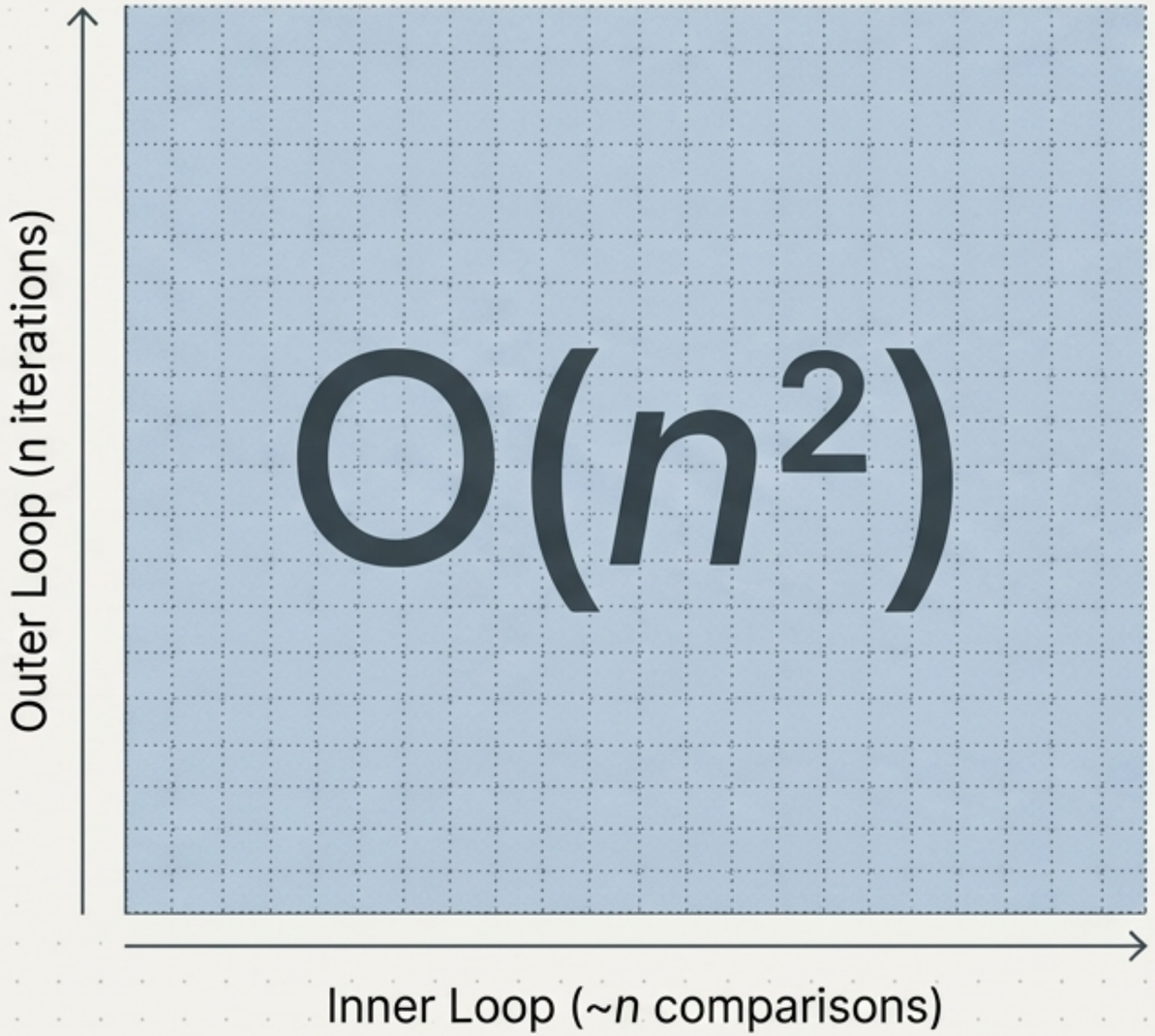
The Shrinking Optimization Window

The Outer Loop (The Repeating Day)

```
def bubble_sort(A):
    n = len(A)
    for i in range(n):
        for j in range(n - 1 - i):
            if A[j] > A[j+1]:
                A[j], A[j+1] = A[j+1], A[j]
```

The Single-Line Python Swap

Nested loops directly dictate the quadratic time complexity



The outer loop executes exactly n times.

The inner loop executes approximately n times per outer loop.

Multiplying nested iterations yields a quadratic computational cost, scaling exponentially as data sets grow larger.